Gestion de versions avec Git et GitHub

Informatique — Licence 2 (L2)

Cours INF-L2-18

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Ne plus jamais perdre son code ni travailler sur des copies « final_v2 » : Git enregistre l'historique complet d'un projet, permet à chacun de travailler sur sa branche, puis fusionne le tout proprement. Ce cours va de l'installation aux pull requests, avec un projet d'équipe mené de bout en bout.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Pourquoi gérer les versions : historique et collaboration
• Leçon 2 — Commandes de base : init, add, commit et log
• Leçon 3 — Branches : créer, fusionner et résoudre les conflits
• Leçon 4 — Travailler avec GitHub : remote, push et pull
• Leçon 5 — Pull requests et relecture de code entre pairs
• Leçon 6 — Bonnes pratiques : messages de commit et fichiers ignorés
• Leçon 7 — Projet collaboratif mené en équipe avec Git
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Repère la scène : la veille de la soutenance, le groupe cherche LE fichier du projet. Sur la clé USB de Moussa, il y a « projet_final.py », « projet_final2.py » et « projet_final_VRAIMENT_final.py ». Lequel contient la correction du bug d'hier soir ? Personne ne sait. Pire : la version la plus récente corrige bien le bug, mais elle a perdu la fonction que Fatou avait ajoutée le matin même, écrasée par une copie plus ancienne. Cette petite catastrophe, presque tous les étudiants en informatique l'ont vécue. Elle a une cause simple : plusieurs personnes modifient les mêmes fichiers, sans outil pour enregistrer l'historique ni pour combiner les travaux. Git a été créé exactement pour faire disparaître ce chaos.

Git est un système de gestion de versions distribué, créé en 2005 par Linus Torvalds pour coordonner le développement du noyau Linux, auquel contribuent des milliers de développeurs. Il est devenu l'outil standard de pratiquement toutes les équipes de développement dans le monde. Son principe : chaque copie du projet contient l'historique complet des modifications, sous forme de commits signés et datés. GitHub, de son côté, est le service en ligne qui héberge ces dépôts et ajoute les outils de collaboration : pull requests, relecture de code, suivi de tâches. Dans ce cours, tu appliqueras Git et GitHub à un fil rouge : BiblioIAM, le petit programme de gestion de bibliothèque que l'équipe L2-Groupe4 de l'IAM — Awa, Moussa et Fatou — construit leçon après leçon.

Chaque leçon suit le même rythme : un peu de théorie, une session de terminal à retaper sur ta propre machine, puis un exercice corrigé. Les sorties de commandes

montrées dans les exemples sont celles que Git produit réellement, messages en anglais compris, car c'est ainsi que tu les verras sur ton écran — le texte qui les entoure les explique en français. Il te faut un poste avec Git installé, un terminal, et un compte gratuit sur github.com. Les acquis des cours de ligne de commande et d'administration système suffisent largement comme prérequis. À la fin du cours, la leçon 7 assemble tout : un projet collaboratif complet, mené en équipe, de la première branche jusqu'à la soutenance.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer ce qu'est un gestionnaire de versions et les problèmes qu'il résout : pertes de fichiers, copies multiples, travail simultané.
- Créer un dépôt local et enregistrer des versions propres avec git init, git add, git commit et git log.
- Distinguer les trois zones de Git : répertoire de travail, index (staging) et dépôt, et lire git status.
- Créer des branches, les fusionner (fast-forward ou commit de fusion) et résoudre un conflit en comprenant les marqueurs <<<<<<<, ======= et >>>>>>>.
- Relier un dépôt local à GitHub : remote, push, clone, fetch et pull — et distinguer clairement fetch de pull.
- Proposer une pull request, relire le code d'un pair et intégrer ses remarques avant la fusion.
- Appliquer les bonnes pratiques d'équipe : messages de commit clairs, fichier .gitignore, branche principale toujours stable, aucun secret dans l'historique.

Leçon 1 — Pourquoi gérer les versions : historique et collaboration

Travailler sans gestionnaire de versions, c'est gérer l'histoire d'un projet à la main : on copie le dossier, on renomme les fichiers « v2 », « v3 », « final », « final_corrigé », et très vite personne ne sait quelle copie fait foi. Si deux membres modifient le même fichier en parallèle, la dernière copie écrase silencieusement le travail de l'autre. Et quand une modification casse le programme, impossible de revenir précisément à l'état d'hier : il faut deviner ce qui a changé, ligne par ligne, de mémoire. Ces problèmes ne sont pas des malchances : ce sont les conséquences prévisibles d'une méthode artisanale appliquée à un travail d'équipe.

Un gestionnaire de versions enregistre des instantanés du projet, appelés commits : chaque commit contient l'état des fichiers, le nom de son auteur, sa date et un message qui explique le changement. L'ensemble forme un historique navigable : on peut voir qui a changé quoi et pourquoi, comparer deux versions ligne par ligne (git diff), et revenir à n'importe quel état passé. Git, créé en 2005 par Linus Torvalds pour le noyau Linux, est dit « distribué » : contrairement aux anciens systèmes centralisés, chaque collaborateur détient une copie complète du dépôt, historique inclus. On peut donc travailler hors ligne, et il n'existe pas de serveur unique dont la perte effacerait le projet.

Pour la collaboration, Git change de logique : au lieu de s'échanger des fichiers en espérant que rien ne s'écrase, chacun travaille sur sa propre ligne de développement — une branche — puis les travaux sont fusionnés de manière raisonnée (leçon 3). GitHub ajoute la couche en ligne : un dépôt distant partagé qui sert de référence commune, les pull requests pour discuter avant d'intégrer (leçon 5), et la relecture de code. Attention à deux idées fausses : Git n'est pas une sauvegarde automatique — rien n'est enregistré tant qu'on n'a pas commité, ni partagé tant qu'on n'a pas poussé ; et il ne sert pas qu'aux gros projets professionnels — un TP de deux semaines en binôme en profite déjà pleinement.

L'essentiel
Un gestionnaire de versions enregistre l'historique complet d'un projet : qui a changé quoi, quand et pourquoi. Git, distribué, donne à chaque collaborateur une copie complète de cet historique ; GitHub l'héberge et organise la collaboration.

Exemple — Le dossier d'un groupe sans Git… et le même projet vu par Git

```
$ ls projet_groupe4/
bibliotheque.py
bibliotheque_v2.py
bibliotheque_v2_corrige_par_moussa.py
bibliotheque_FINAL.py
bibliotheque_FINAL_vraiment_final.py
notes_du_cours.docx
notes_du_cours (copie).docx

# Personne ne sait quelle version envoyer au professeur...

# Avec Git, le meme projet tient en UN seul dossier,
# et son histoire se lit en une commande :
$ git log --oneline --graph
* a502cf2 Ajoute la fonction rendre pour les retours
* 51a5756 Merge branch 'nouveau-titre'
|\
| * 43c0bc3 Renomme le projet en BiblioIAM
* | 53a5c94 Precise le titre : bibliotheque de l IAM
|/
* 8dab11e Ajoute la fonction rechercher
* 75869c0 Ajoute la fonction emprunter
* eb8aac3 Ajoute la structure initiale du projet
```

À retenir

- Un commit est un instantané du projet, identifié par un code unique (le hash), avec un auteur, une date et un message.
- Git est distribué : chaque membre de l'équipe possède l'historique entier du projet, pas seulement la dernière version.
- Git n'est pas GitHub : Git est l'outil local d'historique, GitHub est le service en ligne qui héberge les dépôts et organise la relecture.

Exercice

Un groupe de quatre étudiants prépare un mini-projet. Chacun modifie le fichier programme.py sur son ordinateur, et les versions circulent par clé USB et par WhatsApp. La veille de la remise, ils découvrent trois versions différentes, et aucune ne contient toutes les corrections. Cite trois problèmes précis de cette façon de travailler et, pour chacun, la capacité de Git qui le résout.

✔ Corrigé

1) Il n'existe aucune version de référence : chacun détient une vérité différente. Git résout cela par un dépôt distant unique qui fait foi, duquel tout le monde repart (clone, pull). 2) Les corrections se perdent et s'écrasent : la dernière copie reçue remplace les autres. Avec Git, chaque modification devient un commit attribué à son auteur, et la fusion combine les travaux au lieu de les écraser. 3) Impossible de savoir qui a changé quoi, ni de revenir en arrière après une casse. L'historique Git est daté, signé et navigable : git log montre chaque changement, et l'on peut revenir à n'importe quel commit. Bonus : sans Git, on ne peut pas travailler vraiment en même temps sur le même fichier ; les branches Git rendent ce travail parallèle possible.

Leçon 2 — Commandes de base : init, add, commit et log

Tout commence par une configuration unique : git config --global user.name "Ton Nom" et user.email fixent l'identité qui signera chaque commit — c'est elle qui permet de savoir qui a fait quoi. Ensuite, git init transforme un dossier ordinaire en dépôt Git, en créant le dossier caché .git qui contient tout l'historique. L'option -b main nomme directement la branche principale « main », le nom standard actuel. À ce stade, tout est local : aucun compte ni aucune connexion n'est nécessaire pour versionner sur sa machine.

Le cœur de Git tient en trois zones qu'il faut visualiser clairement. Le répertoire de travail, ce sont les fichiers que tu vois et modifies dans ton éditeur. L'index (en anglais staging area), c'est la zone de préparation : tu y places, avec git add, exactement les modifications qui formeront le prochain commit. Le dépôt, enfin, c'est l'historique : git commit y enregistre définitivement le contenu de l'index. Conséquence fondamentale : git add ne sauvegarde rien, il sélectionne ; c'est git commit qui écrit l'histoire. Un fichier tout nouveau est dit « non suivi » (untracked) tant qu'il n'a pas été ajouté une première fois. La commande git status est le tableau de bord permanent : elle dit, pour chaque fichier, dans quelle zone il se trouve.

Pour lire l'histoire, git log affiche chaque commit avec son hash complet (40 caractères, une empreinte unique calculée à partir du contenu), son auteur, sa date et son message ; git log --oneline condense l'affichage à une ligne par commit, avec les 7 premiers caractères du hash — largement suffisants au quotidien. git show <hash> détaille un commit précis, et git diff montre les différences entre la zone de travail et l'index. Retiens le réflexe des professionnels : git status avant et après chaque opération, jusqu'à ce que la lecture des trois zones devienne automatique.

L'essentiel
Trois zones, trois verbes : tu modifies dans le répertoire de travail, tu sélectionnes avec git add (index), tu enregistres avec git commit (dépôt). git status te dit toujours où tu en es ; git log raconte l'histoire.

Exemple — Naissance du dépôt BiblioIAM : init, status, add, commit, log

```
$ git init -b main gestion-bibliotheque
Initialized empty Git repository in /home/awa/gestion-bibliotheque/.git/

$ cd gestion-bibliotheque
$ git status
On branch main

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)
README.md
bibliotheque.py

nothing added to commit but untracked files present (use "git add" to track)

$ git add README.md bibliotheque.py
$ git status --short
A README.md
A bibliotheque.py

$ git commit -m "Ajoute la structure initiale du projet"
[main (root-commit) eb8aac3] Ajoute la structure initiale du projet
2 files changed, 10 insertions(+)
create mode 100644 README.md
create mode 100644 bibliotheque.py

$ git log --oneline
eb8aac3 Ajoute la structure initiale du projet
```

À retenir

- git add ne sauvegarde rien : il prépare le prochain commit. C'est git commit qui écrit dans l'historique.
- git status est ton meilleur ami : lance-le avant et après chaque opération pour savoir où en sont tes fichiers.
- Un fichier jamais ajouté est « non suivi » (untracked) : Git le voit, mais il n'entrera dans aucun commit tant qu'il n'est pas dans l'index.

Exercice

Dans ton dépôt, tu crées un fichier menu.py, puis tu modifies README.md. Tu lances git add menu.py, puis git commit -m "Ajoute le menu". a) Le commit contient-il la modification de README.md ? Pourquoi ? b) Que montre git status juste après le

commit ? c) Quelle commande aurait permis d'inclure aussi la modification de README.md dans ce commit ?

✔ Corrigé
a) Non. Un commit ne contient que ce qui se trouve dans l'index au moment du commit : git add n'y a placé que menu.py. La modification de README.md est restée dans la zone de travail, non indexée. b) git status affiche README.md dans la section « Changes not staged for commit » (modifié, non préparé), et plus rien sur menu.py, désormais enregistré proprement. c) Il fallait lancer git add README.md (ou git add . pour tout sélectionner) avant le commit. La modification n'est pas perdue : un second commit, après un nouvel add, l'enregistrera.

Leçon 3 — Branches : créer, fusionner et résoudre les conflits

Une branche est une ligne de développement indépendante ; techniquement, c'est un simple pointeur léger qui désigne un commit et avance avec lui. L'intérêt est immédiat : on développe une fonctionnalité sur sa branche, sans jamais risquer de casser la version stable que les autres utilisent. git switch -c ajout-recherche crée la branche et s'y place ; git switch main revient sur main ; git branch (sans argument) liste les branches et marque la courante d'une étoile. La règle d'équipe à retenir dès maintenant : la branche main doit toujours contenir un projet qui fonctionne.

Fusionner (git merge) consiste à ramener le travail d'une branche dans la branche courante. Premier cas, le plus simple : si main n'a reçu aucun commit depuis la création de ta branche, Git se contente d'avancer son pointeur jusqu'au dernier commit de ta branche — c'est la fusion fast-forward, sans aucun commit supplémentaire, comme le montre la première partie de l'exemple. Deuxième cas : les deux branches ont avancé chacune de leur côté. Git effectue alors une fusion à trois voies, en comparant les deux versions à leur ancêtre commun, et crée un commit de fusion qui possède deux parents. Tant que les modifications touchent des endroits différents des fichiers, cette fusion est entièrement automatique.

Le conflit survient quand les deux branches ont modifié les mêmes lignes : Git refuse de choisir à ta place, car il ne sait pas quelle version est la bonne. Il interrompt la fusion, marque le fichier en conflit (état UU dans git status) et y écrit les deux versions entre des marqueurs : <<<<<<< HEAD introduit la version de ta branche courante, ======= sépare, et >>>>>>> nom-de-branche termine avec la version entrante. Résoudre un conflit, c'est éditer le fichier pour écrire la version finale — souvent un mélange des deux —, supprimer les trois marqueurs, puis git add pour déclarer le fichier résolu, et git commit pour conclure la fusion. Un conflit n'est ni une erreur ni une panne : c'est Git qui demande un arbitre humain. Et tant que tu n'as pas commité, git merge --abort annule tout et revient à l'état d'avant la fusion.

L'essentiel
Branche = pointeur vers un commit. Fusion fast-forward si la branche cible n'a pas bougé ; sinon Git crée un commit de fusion. Conflit = mêmes lignes modifiées des deux côtés : à toi d'écrire la version finale, puis git add + git commit.

Exemple — Fusion fast-forward, puis vrai conflit sur le titre du README

```
# 1) Fusion simple : main n'a pas bouge, avance directe (fast-forward)
$ git switch -c ajout-recherche
Switched to a new branch 'ajout-recherche'
# ... Awa code la fonction rechercher, puis git add + git commit ...
$ git switch main
Switched to branch 'main'
$ git merge ajout-recherche
Updating 75869c0..8dab11e
Fast-forward
bibliotheque.py | 6 ++++++
1 file changed, 6 insertions(+)

# 2) Conflit : main et la branche nouveau-titre changent
# en meme temps la premiere ligne de README.md
$ git merge nouveau-titre
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.

# Contenu de README.md juste apres le conflit :
<<<<<<< HEAD
# Gestion de la bibliotheque de l IAM
=======
# BiblioIAM - Gestion de bibliotheque
>>>>>>> nouveau-titre

# Awa ecrit le titre final, supprime les marqueurs, puis :
$ git add README.md
$ git commit --no-edit
[main 51a5756] Merge branch 'nouveau-titre'
```

À retenir

- Fast-forward ou commit de fusion : dans les deux cas, après git merge, ta branche courante contient tout le travail de la branche fusionnée.
- <<<<<<< HEAD désigne TA branche courante ; >>>>>>> nom désigne la branche que tu es en train de fusionner.
- Un conflit se termine toujours par : éditer le fichier, git add, git commit. Et git merge --abort permet de tout annuler avant la résolution.

Exercice

Moussa et Fatou modifient tous les deux la fonction afficher_livres() de bibliotheque.py, chacun sur sa propre branche, en touchant les mêmes lignes. Moussa fusionne sa branche dans main sans aucun problème. a) Que se passera-t-il

vraisemblablement quand Fatou fusionnera la sienne ? b) Décris les étapes qu'elle devra suivre, dans l'ordre. c) Que se serait-il passé si leurs modifications avaient touché des fonctions différentes du même fichier ?

✔ Corrigé
a) Un conflit est très probable : depuis l'ancêtre commun, les mêmes lignes ont été modifiées des deux côtés (la branche de Moussa, désormais dans main, et celle de Fatou). Git affichera « CONFLICT (content) » et interrompra la fusion. b) Étapes : ouvrir bibliotheque.py ; repérer les blocs entre <<<<<<< HEAD et >>>>>>> (version de main contre version de sa branche) ; écrire la version finale de la fonction en combinant les deux travaux ; supprimer les trois marqueurs ; tester que le programme fonctionne ; git add bibliotheque.py pour marquer le conflit comme résolu ; git commit pour conclure la fusion. c) Si les modifications portaient sur des fonctions différentes, la fusion à trois voies les aurait combinées automatiquement, avec un simple commit de fusion et aucun marqueur.

Leçon 4 — Travailler avec GitHub : remote, push et pull

GitHub héberge en ligne une copie de ton dépôt : c'est le dépôt distant, qu'on appelle par convention origin. Après avoir créé un dépôt vide sur github.com, on relie le dépôt local au distant une seule fois avec git remote add origin <url> ; git remote -v vérifie les adresses enregistrées, une pour fetch et une pour push. Deux protocoles coexistent : HTTPS, le plus simple pour débuter (GitHub demande alors un jeton d'accès personnel plutôt que le mot de passe du compte), et SSH, basé sur une paire de clés, préféré des utilisateurs réguliers. Dans les deux cas, les commandes Git qui suivent sont identiques.

Publier son travail se dit pousser : git push -u origin main envoie tes commits vers GitHub et crée au passage le suivi de branche — le lien durable entre ta branche locale main et sa copie distante origin/main. Grâce à ce suivi, les fois suivantes, git push et git pull sans argument suffisent. Dans l'autre sens, récupérer un projet existant se fait une seule fois avec git clone <url> : la commande copie le dépôt entier, historique compris, et configure origin automatiquement. C'est ainsi que chaque membre de l'équipe obtient le projet : fini les échanges de fichiers par messagerie, tout le monde pousse et tire depuis la même source.

Récupérer le travail des autres demande de bien distinguer deux commandes souvent confondues. git fetch télécharge les nouveaux commits du distant et met à jour origin/main, ta copie de contrôle de l'état distant — mais il ne touche ni à tes fichiers ni à ta branche : c'est une opération sans risque, que tu peux suivre d'un git log HEAD..origin/main pour inspecter ce qui arrive avant de l'accepter. git pull, lui, vaut fetch + merge : il télécharge puis fusionne immédiatement dans ta branche courante. Il peut donc provoquer un conflit si tu as modifié les mêmes lignes que les commits reçus. Le réflexe sain : travaille par petits commits, et intègre régulièrement le travail distant au lieu d'attendre la fin du projet.

L'essentiel
origin = le nom de ton dépôt distant. push envoie tes commits, clone copie un dépôt entier, fetch regarde ce qui est arrivé sans rien fusionner, pull = fetch + merge. fetch est sans risque ; pull modifie ta branche.

Exemple — Relier BiblioIAM à GitHub, pousser, puis récupérer le commit de Moussa

```
# Relier le depot local au depot GitHub (une seule fois)
$ git remote add origin https://github.com/awa-diallo/gestion-bibliotheque.git
$ git remote -v
origin https://github.com/awa-diallo/gestion-bibliotheque.git (fetch)
origin https://github.com/awa-diallo/gestion-bibliotheque.git (push)

# Premier envoi : -u memorise le lien main <-> origin/main
$ git push -u origin main
To https://github.com/awa-diallo/gestion-bibliotheque.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.

# Plus tard, Moussa a pousse un commit. Awa regarde d'abord...
$ git fetch origin
From https://github.com/awa-diallo/gestion-bibliotheque
51a5756..a502cf2 main -> origin/main
$ git log --oneline HEAD..origin/main
a502cf2 Ajoute la fonction rendre pour les retours

# ...puis elle fusionne dans sa branche avec pull
$ git pull
Updating 51a5756..a502cf2
Fast-forward
bibliotheque.py | 4 ++++
1 file changed, 4 insertions(+)
```

À retenir

- git push -u origin main la première fois ; ensuite git push et git pull suffisent, grâce au suivi de branche.
- fetch ne modifie jamais tes fichiers : il met à jour origin/main, la copie de contrôle du distant, que tu peux inspecter avant de fusionner.
- pull = fetch + merge : si ton travail local touche les mêmes lignes que les commits reçus, un conflit peut apparaître, exactement comme pour un merge local.

Exercice

Fatou allume son ordinateur lundi matin. Pendant le week-end, main a reçu trois commits sur GitHub. De son côté, Fatou possède un commit local non poussé, sur une fonctionnalité qui touche d'autres fichiers. a) Que fait exactement git fetch sur sa machine ? b) Que fait git pull en plus ? c) Pourquoi le risque de conflit est-il faible dans sa situation ?

✔ Corrigé
a) git fetch télécharge les trois commits et met à jour origin/main ; ni ses fichiers de travail ni sa branche locale ne changent. Elle peut inspecter ce qui arrive avec git log HEAD..origin/main. b) git pull fait le même téléchargement, puis fusionne immédiatement origin/main dans sa branche : Git combine son commit local et les trois commits reçus, en créant au besoin un commit de fusion. c) Le risque est faible parce que les modifications portent sur des fichiers différents : la fusion à trois voies les combine automatiquement. Un conflit n'apparaît que lorsque les deux côtés ont modifié les mêmes lignes depuis l'ancêtre commun.

Leçon 5 — Pull requests et relecture de code entre pairs

La pull request — PR en abrégé — est la proposition, faite sur GitHub, de fusionner une branche dans une autre, presque toujours dans main. Elle transforme la fusion en conversation : au lieu de mélanger le code d'abord et de discuter ensuite, l'équipe discute d'abord et fusionne ensuite. Une PR affiche un titre, une description rédigée par l'auteur, la liste des commits concernés et surtout le diff complet, c'est-à-dire toutes les lignes ajoutées et retirées. C'est ce diff que les relecteurs vont examiner. Ouvrir une PR ne demande que quelques clics après avoir poussé sa branche : GitHub propose le bouton « Compare & pull request » dès qu'une branche récente existe.

La relecture est le cœur du dispositif. Un pair lit le diff ligne par ligne et peut commenter une ligne précise : une question (« que se passe-t-il si la liste est vide ? »), une suggestion (« cette boucle existe déjà dans rechercher(), on peut la réutiliser »), ou un problème réel. Il conclut par « Comment » (simple remarque), « Approve » (j'approuve) ou « Request changes » (je demande des corrections avant fusion). L'auteur répond aux commentaires et pousse des commits correctifs sur la même branche : la PR se met à jour automatiquement, inutile d'en ouvrir une nouvelle. Les bénéfices dépassent la chasse aux erreurs : la relecture partage la connaissance du code dans toute l'équipe et fait progresser l'auteur comme le relecteur. Le ton compte autant que le fond : on questionne le code, jamais la personne, et on propose une alternative quand on signale un problème.

Quand la PR est approuvée, la fusion se fait sur GitHub (bouton Merge), ce qui crée le commit de fusion dans main ; la branche de travail peut alors être supprimée, elle a rempli son rôle. GitHub permet aussi de protéger main : exiger au moins une approbation avant toute fusion, par exemple. En local, on peut préparer et vérifier une PR avant de l'ouvrir : git log main..ma-branche liste les commits qu'elle contient, et git diff main...ma-branche --stat résume les fichiers touchés. La règle adoptée dans ce cours, et dans la plupart des équipes : aucune fusion dans main sans la relecture d'au moins un pair.

L'essentiel
La pull request transforme la fusion en conversation : description, diff, commentaires, corrections, approbation, puis fusion. Elle protège la branche main et fait progresser toute l'équipe.

Exemple — Préparer une pull request en local, puis la décrire sur GitHub

```
# Avant d'ouvrir la pull request, verifier ce qu'elle contient :
$ git switch ajout-retours
$ git log --oneline main..ajout-retours
a502cf2 Ajoute la fonction rendre pour les retours
$ git diff main...ajout-retours --stat
bibliotheque.py | 4 ++++
1 file changed, 4 insertions(+)
$ git push -u origin ajout-retours

# Sur GitHub : bouton "Compare & pull request", puis la description :
Titre : Ajoute la gestion des retours de livres

## Quoi
Nouvelle fonction rendre(titre) dans bibliotheque.py :
elle affiche la confirmation du retour d'un livre.

## Pourquoi
Le cahier des charges demande de tracer les retours
en plus des emprunts (voir l'issue n.3 du projet).

## Comment tester
1. Lancer : python bibliotheque.py
2. Appeler rendre("Python facile")
3. Verifier l'affichage : "Retour enregistre : Python facile"

## Relecture demandee
@awa-diallo : la logique metier
@fatou-ndiaye : les messages affiches aux utilisateurs
```

À retenir

- Une PR se décrit comme un courrier à l'équipe : quoi, pourquoi, comment tester. Sans description, la relecture est aveugle.
- Les commits correctifs poussés sur la branche mettent à jour la PR automatiquement : inutile d'en ouvrir une deuxième.
- Relire, ce n'est pas chercher la faute : c'est vérifier la logique, la lisibilité et les cas limites — et apprendre le code des autres.

Exercice

Moussa ouvre une pull request intitulée « Modifs », sans aucune description. Elle contient 14 fichiers modifiés, dont la suppression du fichier .gitignore et un mot de passe écrit en clair dans le code. Tu es le relecteur. Écris les trois commentaires de relecture les plus importants, formulés de manière constructive, et conclus : approbation ou demande de modifications ? Justifie.

✔ Corrigé
Commentaire 1 (description) : « Peux-tu décrire ce que fait cette PR et comment la tester ? Avec 14 fichiers modifiés, je ne sais pas par où commencer ; on gagnerait peut-être à la découper en deux PR plus petites. » Commentaire 2 (sécurité, bloquant) : « Attention, un mot de passe est écrit en clair dans le code : il faut le retirer de la PR, et comme tout ce qui entre dans l'historique y reste, il faudra aussi changer ce mot de passe partout où il est utilisé. » Commentaire 3 (.gitignore) : « La suppression du .gitignore va faire réapparaître __pycache__ et les fichiers personnels dans les prochains commits : est-ce voulu ? Sinon, peux-tu le restaurer ? » Conclusion : demande de modifications (Request changes), pas d'approbation — le secret en clair est un motif bloquant à lui seul.

Leçon 6 — Bonnes pratiques : messages de commit et fichiers ignorés

Le message de commit est la documentation vivante de l'historique : dans trois mois, c'est lui qui expliquera pourquoi le code est ainsi. Un bon message tient en un titre court — environ 50 à 72 caractères — qui commence par un verbe d'action et nomme l'objet du changement : « Ajoute la fonction rendre », « Corrige le calcul des dates », « Supprime l'ancien menu ». Le pourquoi, quand il n'est pas évident, va dans le corps du message, séparé du titre par une ligne vide. À l'inverse, « modif », « test2 » ou « ça marche enfin » ne disent rien et rendront tout retour en arrière hasardeux. Deux disciplines accompagnent le message : un commit ne contient qu'un seul changement logique cohérent (pas cinq sujets mélangés), et on évite de committer sur main un code qui ne fonctionne pas.

Le fichier .gitignore, placé à la racine du dépôt, liste les motifs de fichiers que Git doit ignorer : les fichiers générés (__pycache__/, les .pyc en Python, les .class en Java), les dépendances réinstallables (venv/, node_modules/), les fichiers personnels (notes, brouillons) et surtout les fichiers de secrets (.env, mots de passe). Deux précisions importantes : le .gitignore n'efface rien du disque, il empêche seulement le suivi futur ; et il n'a aucun effet sur un fichier déjà suivi, qui le reste tant qu'on ne le retire pas de l'index. Comme le montre l'exemple, dès que le .gitignore de BiblioIAM est ajouté, git status ne liste plus ni notes-perso.txt ni le dossier __pycache__. Le .gitignore se committe comme les autres fichiers : toute l'équipe partage ainsi les mêmes règles d'hygiène.

Trois habitudes complètent ces bonnes pratiques. D'abord, committer souvent et petit : un commit par étape terminée, et l'on ne perd jamais plus d'une fraction d'heure de travail. Ensuite, ne jamais committer de secret : un mot de passe entré dans l'historique y reste accessible même si un commit suivant l'efface — il faut alors considérer le secret comme compromis et le changer. Enfin, tirer (pull) avant de pousser, pour intégrer le travail des autres tant qu'il est frais. Un mot sur git rebase, que tu croiseras partout : là où merge conserve l'histoire réelle des branches en créant un commit de fusion, rebase réécrit l'historique en replaçant tes commits sur une base plus récente, ce qui donne un parcours linéaire. En équipe débutante, retiens la règle sûre : merge pour intégrer le travail partagé, et jamais de rebase sur une branche déjà

poussée que d'autres utilisent.

L'essentiel
Un bon message de commit explique le changement à un lecteur futur. Le .gitignore garde hors du dépôt tout ce qui est généré, personnel ou secret. Petits commits, main stable, aucun secret dans l'historique.

Exemple — Le .gitignore de BiblioIAM, et des messages de commit avant/après

```
# ----- Fichier .gitignore du projet BiblioIAM -----
# Fichiers generes par Python a l'execution
__pycache__/
*.pyc

# Environnement virtuel (lourd, recreable avec pip)
venv/

# Fichiers personnels de chaque membre
notes-perso.txt
*.local

# Secrets : jamais dans le depot !
.env

# Apres l'ajout de ce fichier, verification :
$ git status --short
?? .gitignore
# notes-perso.txt et __pycache__/ ont disparu de la liste :
# ils existent toujours sur le disque, mais Git les ignore.

# ----- Messages de commit : le mauvais et le bon -----
# NON : "modif", "test2", "ca marche", "aaaa"
# OUI : "Ajoute la fonction rendre pour les retours"
# OUI : "Corrige l'affichage des titres avec accents"
# OUI : "Supprime l'ancien fichier de tests en doublon"
```

À retenir

- Message de commit : un verbe d'action + l'objet du changement. Le « pourquoi » va dans le corps du message, pas dans le titre.
- Le .gitignore n'efface rien : il empêche Git de suivre les fichiers générés, personnels ou secrets — il doit être committé dès le début du projet.
- merge conserve l'histoire (commit de fusion), rebase la réécrit (historique linéaire) : ne rebase jamais une branche déjà partagée.

Exercice

a) Pour chaque message, dis s'il est acceptable et réécris ceux qui ne le sont pas : « correction » ; « Ajoute le tri des livres par titre » ; « Modifie bibliotheque.py, README.md, les tests, et au fait la page d'accueil aussi » ; « version finale ». b) Écris les motifs de .gitignore pour ignorer : le dossier __pycache__, tous les fichiers se

terminant par .log, et le fichier .env.

✔ Corrigé
a) « correction » : non, trop vague — on ne sait ni quoi ni où ; réécriture : « Corrige le calcul de la date de retour » (le principe : préciser l'objet corrigé). « Ajoute le tri des livres par titre » : oui, verbe d'action, objet précis, un seul sujet. Le troisième message : non — il mélange au moins trois changements indépendants ; il faut découper en trois ou quatre commits distincts, chacun avec son message. « version finale » : non — il ne décrit aucun changement ; réécrire en décrivant ce que la version apporte réellement, par exemple « Ajoute la recherche de livres par mot-cle ». b) Les motifs sont : __pycache__/ (le slash final désigne un dossier), *.log (tous les fichiers de journal), .env (le fichier de secrets lui-même).

Leçon 7 — Projet collaboratif mené en équipe avec Git

Le projet final assemble toutes les leçons en un flux de travail d'équipe complet. D'abord, l'organisation : l'équipe répartit les rôles et les tâches — pour BiblioIAM, Awa s'occupe du catalogue des livres, Moussa des emprunts et des retours, Fatou du menu et de la documentation — et écrit ses règles dans le README : une branche par tâche, pull request obligatoire pour entrer dans main, main toujours présentable. Ces règles, convenues avant de coder, évitent la plupart des discussions et des catastrophes de dernière minute. Chacun clone le dépôt, vérifie son identité Git (user.name, user.email) et s'assure que le .gitignore est en place dès le premier jour.

Le rituel quotidien, identique pour chaque membre et chaque tâche, tient en sept gestes : partir d'un main à jour (git switch main, git pull) ; créer sa branche de tâche (git switch -c tache-...) ; coder par petits commits au message clair ; pousser sa branche sur GitHub ; ouvrir la pull request avec une vraie description ; intégrer les remarques de la relecture croisée et laisser fusionner ; enfin, revenir sur main, tirer la version fusionnée et supprimer la branche terminée. Ce rituel rend la collaboration prévisible : à tout moment, chacun peut dire où en est chaque tâche en regardant les branches et les PR ouvertes, sans réunion ni échange de fichiers.

Restent les imprévus, qui font partie de tout projet réel. Si deux branches vivent longtemps en parallèle, elles divergent et la fusion finale devient douloureuse : la parade est de fusionner main dans sa propre branche régulièrement, pour absorber les écarts en petites doses. Si un membre est absent, son travail n'est pas prisonnier de son ordinateur : ses branches poussées sur GitHub permettent à un autre de continuer. Le jour de la soutenance, on gèle les fusions, on vérifie la version finale de main de bout en bout, et l'on présente au jury le graphe de l'historique (git log --graph --oneline --all) : il prouve, auteurs et dates à l'appui, le travail réel de chaque membre de l'équipe.

L'essentiel
Le rituel d'équipe : pull, branche, petits commits, push, pull request, relecture, fusion, re-pull. Répété par chacun à chaque tâche, il rend la collaboration prévisible et sans perte.

Exemple — Une semaine de projet BiblioIAM : le rituel complet et l'histoire finale

```
# Le rituel de chaque membre, pour chaque tache :
$ git switch main && git pull # 1. partir a jour
$ git switch -c tache-menu # 2. une branche par tache
# ... Fatou code le menu, par petits commits ...
$ git add menu.py
$ git commit -m "Ajoute le menu principal"
$ git commit -m "Ajoute le choix 3 : rechercher un livre"
$ git push -u origin tache-menu # 3. publier la branche
# 4. Pull request sur GitHub : relecture par Awa,
# corrections, approbation, fusion dans main.
$ git switch main && git pull # 5. recuperer le travail fusionne
$ git branch -d tache-menu # 6. nettoyer la branche terminee

# L'histoire du projet BiblioIAM (vue partielle) :
$ git log --graph --oneline
* a502cf2 Ajoute la fonction rendre pour les retours
* 51a5756 Merge branch 'nouveau-titre'
|\
| * 43c0bc3 Renomme le projet en BiblioIAM
* | 53a5c94 Precise le titre : bibliotheque de l IAM
|/
* 8dab11e Ajoute la fonction rechercher
* 75869c0 Ajoute la fonction emprunter
* eb8aac3 Ajoute la structure initiale du projet
```

À retenir

- Une branche par tâche, une pull request par branche : jamais de développement direct sur main quand on travaille en équipe.
- Fusionner main dans sa branche régulièrement évite le « conflit géant » de la veille de la remise.
- Le graphe git log --graph --oneline --all est la preuve visuelle du travail collectif : montrez-le au jury de soutenance.

Exercice

Le matin de la soutenance, Fatou découvre que sa branche tache-menu, créée il y a cinq jours, est très en retard : main a huit commits d'avance, et la fusion provoque trois fichiers en conflit. La remise est à 14 h. Propose un plan d'action ordonné pour livrer à temps sans perdre ni son travail ni celui des autres, puis dis ce que l'équipe aurait pu faire chaque jour pour éviter cette situation.

✔ Corrigé
Plan d'action : 1) Sécuriser — pousser immédiatement sa branche sur GitHub (git push) : son travail est sauvegardé hors de sa machine, quoi qu'il arrive ensuite. 2) Ne pas fusionner dans main sous la panique : fusionner main DANS sa branche tache-menu, en local, là où un échec ne casse rien pour les autres. 3) Résoudre les trois conflits un fichier à la fois, en appelant l'auteur des commits concernés quand un choix est ambigu, et tester le programme après chaque fichier résolu. 4) Committer la fusion, pousser, ouvrir la PR avec une relecture éclair mais réelle par Awa. 5) Fusionner dans main, faire tirer la version finale par toute l'équipe, tester le projet complet, puis geler main jusqu'à la soutenance. Prévention au quotidien : partir chaque matin d'un main à jour (pull), fusionner main dans sa branche tous les jours, garder des branches courtes (une à deux journées maximum) et annoncer qui touche quel fichier — les conflits se règlent alors en quelques lignes, jamais en trois fichiers d'un coup.

Travaux dirigés (TD)

TD 1 — Lire et interroger l'historique du projet BiblioIAM

Voici la sortie de git log --graph --oneline obtenue sur le projet BiblioIAM : * a502cf2 Ajoute la fonction rendre pour les retours * 51a5756 Merge branch 'nouveau-titre' |\ | * 43c0bc3 Renomme le projet en BiblioIAM * | 53a5c94 Precise le titre : bibliotheque de l IAM |/ * 8dab11e Ajoute la fonction rechercher * 75869c0 Ajoute la fonction emprunter * eb8aac3 Ajoute la structure initiale du projet Réponds aux questions : a) Combien de commits compte ce dépôt, et lequel est le plus ancien ? b) Le commit 51a5756 est un commit de fusion : quels sont ses deux parents, et quel est l'ancêtre commun des deux branches fusionnées ? c) Avant ce graphe, la branche ajout-recherche avait été fusionnée avec la sortie suivante : « Updating 75869c0..8dab11e / Fast-forward ». De quel type de fusion s'agit-il, et pourquoi aucun commit de fusion n'apparaît-il pour elle dans le graphe ? d) Sur une autre machine, un étudiant a produit les messages « maj truc » et « fin (pour de vrai) ». Réécris-les en messages acceptables, en inventant un contenu plausible lié à BiblioIAM.

✔ Corrigé du TD 1
a) Le dépôt compte 7 commits. Le plus ancien est le dernier de la liste : eb8aac3 « Ajoute la structure initiale du projet » — git log affiche toujours du plus récent au plus ancien. b) Les deux parents de 51a5756 sont 53a5c94 (le commit de main, « Precise le titre ») et 43c0bc3 (le commit de la branche nouveau-titre, « Renomme le projet en BiblioIAM »). Leur ancêtre commun est 8dab11e « Ajoute la fonction rechercher », le dernier commit partagé avant la divergence. c) C'est une fusion fast-forward : main n'avait reçu aucun commit depuis la création de la branche (elle pointait encore sur 75869c0), Git a donc simplement avancé le pointeur de main jusqu'à 8dab11e. Aucun commit de fusion n'est créé dans ce cas, l'histoire reste linéaire — c'est pour cela que 8dab11e apparaît comme un commit ordinaire. d) « maj truc » ne dit ni quoi ni où : réécriture plausible, « Ajoute la fonction rendre pour les retours » (c'est d'ailleurs le vrai message du commit a502cf2). « fin (pour de vrai) » décrit un espoir, pas un changement : réécriture plausible, « Corrige l'affichage du titre dans le README » — un verbe d'action, un objet précis, un seul sujet.

TD 2 — Sauver un groupe en désordre : la migration vers Git en une soirée

Le groupe 7 travaille depuis trois semaines sur un projet de gestion de stock. Leur dossier partagé contient quatre versions du programme, la plus récente a été écrasée hier par une ancienne, et personne ne sait plus qui a corrigé le bug des quantités négatives. Ils décident de passer à Git ce soir, avant la dernière semaine de travail. Rédige le plan complet de la migration : 1) les précautions à prendre AVANT de toucher à quoi que ce soit ; 2) la création du dépôt local et le choix du contenu du premier commit ; 3) le fichier .gitignore à prévoir pour ce projet Python (dossier __pycache__, fichier notes.txt personnel, fichier .env de configuration) ; 4) la mise en ligne sur GitHub et la manière dont chaque membre récupère le projet ; 5) trois règles de fonctionnement d'équipe à adopter pour la dernière semaine.

✔ Corrigé du TD 2

1) Précautions : d'abord, ne rien effacer. Copier le dossier complet dans un endroit sûr (sauvegarde datée), puis comparer les quatre versions (avec git diff --no-index ou à la main) pour reconstruire UNE version de référence qui contient toutes les corrections connues, dont celle du bug des quantités négatives ; c'est elle, et elle seule, qui entrera dans Git. 2) Dépôt local : dans le dossier de référence, git init -b main, configurer son identité, créer le .gitignore AVANT le premier add, puis git add . et un premier commit au message honnête : « Importe la version de reference du projet (avant Git) ». L'historique commence proprement, sans les quatre versions concurrentes. 3) .gitignore : les lignes __pycache__/, notes.txt et .env (avec des commentaires pour expliquer chaque motif). 4) Mise en ligne : créer un dépôt vide sur github.com (sans README automatique, le projet en a déjà un), git remote add origin <url>, git push -u origin main. Chaque membre récupère ensuite le projet par git clone <url> — jamais en copiant le dossier d'un autre, sinon les dépôts ne seraient pas reliés. 5) Règles pour la dernière semaine : une branche par tâche avec pull request obligatoire relue par un pair ; main toujours fonctionnelle et gelée la veille de la remise ; pull chaque matin avant de commencer et petits commits au message clair.

Travaux pratiques (TP)

TP 1 — Mener BiblioIAM en équipe de trois pendant une semaine

Par équipe de trois, vous reprenez le projet BiblioIAM (ou un équivalent : gestion de stock, de notes, de cantine) et vous le faites avancer pendant cinq séances en appliquant tout le cours. Répartition minimale : un catalogue de données (liste d'objets), deux fonctionnalités métier (ajout/recherche, emprunt/retour, statistiques…), un menu ou une interface simple, et la documentation. Contraintes obligatoires : dépôt hébergé sur GitHub ; fichier .gitignore présent dès le premier commit ; une branche par tâche et par personne ; au moins trois pull requests relues et commentées par un pair différent de l'auteur ; aucun commit direct sur main après le commit initial ; messages de commit au format « Verbe + objet ». Le dernier jour, chaque membre doit pouvoir expliquer l'historique complet du projet à partir de git log

--graph.

✔ Attendus du TP 1
Attendus vérifiables par l'enseignant : (1) le dépôt GitHub est accessible et son historique montre un commit initial propre (structure + .gitignore + README nommant les membres et leurs rôles) ; (2) git log --graph --oneline --all fait apparaître au moins trois branches fusionnées, avec des commits de fusion issus des pull requests ; (3) chaque pull request possède une description (quoi / pourquoi / comment tester) et au moins un commentaire de relecture signé par un autre membre, suivi le cas échéant de commits correctifs sur la même branche ; (4) les messages de commit respectent le format annoncé et aucun ne ressemble à « modif » ou « final » ; (5) git status sur une copie fraîchement clonée est propre, et les fichiers ignorés (__pycache__, notes personnelles, .env) n'apparaissent dans aucun commit (vérifiable avec git log --name-only) ; (6) la branche main contient un programme qui se lance sans erreur. Barème indicatif : historique et branches 30 %, qualité des pull requests et relectures 30 %, messages et hygiène (.gitignore, absence de secrets) 20 %, programme final fonctionnel sur main 20 %.

Exercices corrigés supplémentaires

Exercice 1

Après un git add bibliotheque.py, tu modifies ENCORE bibliotheque.py dans ton éditeur, puis tu lances git commit -m "Ajoute rechercher". a) Quelle version de bibliotheque.py contient le commit : celle du moment du add, ou la toute dernière ? b) Que montre git status juste après le commit, pour ce fichier ? c) Que fallait-il faire pour que la dernière modification soit incluse ?

✔ Corrigé
a) Le commit contient la version du moment du git add : commit enregistre le contenu de l'index, pas celui du disque. La modification postérieure est restée dans la zone de travail. b) git status montre bibliotheque.py comme « modified » (changements non indexés) : le fichier est à la fois à jour dans l'historique (ancienne version) et modifié sur le disque — c'est la situation classique du fichier présent dans deux zones à la fois. c) Il fallait relancer git add bibliotheque.py après la modification, avant de committer : chaque add photographie l'état du fichier à cet instant. Réflexe : git status (ou git diff --staged pour voir ce qui partira dans le commit) juste avant chaque commit.

Exercice 2

Tu travailles hors ligne dans le car et tu fais deux commits sur ta branche. En arrivant au campus, tu découvres qu'un pair a poussé deux commits sur la même branche distante, touchant les mêmes fichiers mais d'autres fonctions. a) Pourquoi ton git push est-il refusé ? Cite le terme technique. b) Décris la séquence complète pour publier ton travail sans écraser le sien. c) À quel moment précis un conflit peut-il apparaître dans cette séquence ?

✔ Corrigé
a) Le push est refusé parce qu'il ne serait pas un fast-forward pour la branche distante : le distant possède des commits que tu n'as pas (« non-fast-forward », GitHub affiche « rejected »). Git protège ainsi le travail déjà publié contre l'écrasement. b) Séquence : git fetch origin pour télécharger les commits du pair dans origin/main (ou origin/ma-branche) ; inspecter avec git log HEAD..origin/main ; puis git pull — ou git merge origin/main — pour fusionner les commits reçus dans ta branche ; résoudre les éventuels conflits, tester ; enfin git push, qui devient possible car ta branche contient désormais tout l'historique distant plus tes commits. c) Le conflit ne peut apparaître qu'au moment de la fusion (pendant le pull/merge), et seulement si les deux côtés ont modifié les mêmes lignes ; ici, les fonctions touchées sont différentes, la fusion a donc de bonnes chances d'être automatique.

Exercice 3

Voici quatre messages de commit relevés dans le dépôt d'un groupe : « trucs », « Correction du bug (enfin) », « v3 », « Ajoute la fonction ». a) Classe-les du moins bon au meilleur et justifie en une phrase chacun. b) Réécris le plus mauvais et l'avant-dernier en messages acceptables, dans le contexte d'un projet de gestion de bibliothèque. c) Le projet passe de Python à Java : écris les motifs de .gitignore pour les fichiers compilés .class et le dossier bin/.

✔ Corrigé
a) Du moins bon au meilleur : « trucs » (ne dit absolument rien, ni action ni objet) ; « v3 » (un numéro de version ne décrit pas le changement) ; « Correction du bug (enfin) » (l'action est là mais l'objet manque : quel bug ? la mention « enfin » n'apporte rien) ; « Ajoute la fonction » (verbe et action clairs, mais objet incomplet : quelle fonction ?). Aucun n'est pleinement acceptable, le dernier est le plus proche du format visé. b) « trucs » → par exemple « Ajoute les fonctions emprunter et rendre dans bibliotheque.py ». « Correction du bug (enfin) » → « Corrige le calcul de la date de retour des emprunts ». c) Les motifs sont *.class (tous les fichiers compilés Java) et bin/ (le dossier de sortie, avec le slash final qui désigne un répertoire).

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quel est le rôle de l'index (staging area) dans Git ?

   A) Sauvegarder automatiquement le projet sur GitHub

   B) Sélectionner les modifications qui entreront dans le prochain commit

   C) Compresser l'historique pour faire de la place

2. Quelle commande crée un dépôt Git dans un dossier ?

   A) git init

   B) git start

   C) git new-repo

3. Dans un fichier en conflit, le texte situé entre <<<<<<< HEAD et ======= correspond à…

A) la version la plus ancienne du projet

B) la version de la branche que l'on reçoit

C) la version de ta branche courante

4. git fetch, utilisé seul, a pour effet de…

A) fusionner les nouveautés dans ta branche courante

B) télécharger les commits distants sans modifier tes fichiers de travail

C) envoyer tes commits sur GitHub

5. Une fusion est dite fast-forward quand…

A) la branche cible n'a reçu aucun commit depuis la création de l'autre branche : Git avance simplement son pointeur

B) Git crée automatiquement deux commits de fusion

C) les deux branches ont modifié exactement les mêmes lignes

6. Pour intégrer en local le travail d'une branche dans main, la commande est…

A) git push main

B) git clone nom-de-branche

C) git merge nom-de-branche, lancé depuis main

7. Lequel de ces messages de commit est le meilleur ?

A) « modif »

B) « Ajoute la fonction rendre pour les retours »

C) « Voici tout le code que j'ai tapé aujourd'hui, voir les fichiers »

8. Le fichier .gitignore sert à…

A) empêcher Git de suivre les fichiers générés, personnels ou secrets

B) supprimer définitivement les fichiers du disque dur

C) rendre le dépôt invisible sur GitHub

9. Quelle est la vraie différence entre merge et rebase ?

A) merge envoie le code sur GitHub, rebase le télécharge

B) merge ne fonctionne que sur les fichiers texte, rebase que sur le code

C) merge conserve l'histoire réelle des branches avec un commit de fusion ; rebase réécrit l'historique en le rendant linéaire

10. Dans une pull request, « Request changes » signifie que…

A) la pull request est supprimée définitivement

B) le relecteur demande des corrections avant d'approuver la fusion

C) GitHub fusionne immédiatement la branche

Corrigés du QCM

1. Réponse B — L'index est la zone de préparation située entre la zone de travail et le dépôt : git add y place ce que git commit enregistrera.

2. Réponse A — git init crée le dossier caché .git qui transforme le dossier en dépôt ; « git start » et « git new-repo » n'existent pas.

3. Réponse C — HEAD désigne la branche sur laquelle tu te trouves pendant la fusion ; la version entrante se trouve après =======, avant >>>>>>>.

4. Réponse B — fetch met à jour la copie de contrôle origin/main et s'arrête là : la fusion n'a lieu qu'avec un merge explicite ou via pull.

5. Réponse A — Fast-forward = avance directe du pointeur, sans commit de fusion ; elle n'est possible que si la cible n'a pas divergé.

6. Réponse C — On se place sur la branche qui reçoit (main), puis git merge désigne la branche à intégrer. push sert au distant, clone à copier un dépôt.

7. Réponse B — Verbe d'action + objet précis + un seul sujet : c'est le format qui rend l'historique lisible et les retours en arrière possibles.

8. Réponse A — Il liste des motifs de fichiers à ne pas suivre (ex. __pycache__/, .env) ; les fichiers restent sur le disque et le dépôt reste visible normalement.

9. Réponse C — merge et rebase intègrent tous les deux des commits, mais merge conserve la trace des branches alors que rebase réécrit les commits sur une nouvelle base — d'où l'interdiction de rebaser une branche partagée.

10. Réponse B — C'est l'une des trois conclusions d'une relecture, avec « Comment » et « Approve » : l'auteur corrige sur la même branche, la PR se met à jour, puis la relecture reprend.

Résumé — les notions importantes

• Git enregistre l'historique complet d'un projet sous forme de commits identifiés par un hash, avec auteur, date et message : on peut toujours revenir en arrière et savoir qui a fait quoi.

• Trois zones à ne jamais confondre : répertoire de travail, index (git add), dépôt (git commit). git status décrit l'état de chaque fichier entre ces zones.

• Une branche est un pointeur léger vers un commit : on y développe sans risquer la branche principale main, qui doit rester stable et fonctionnelle.

• Fusion : fast-forward si la cible n'a pas bougé, sinon commit de fusion. Mêmes lignes modifiées des deux côtés = conflit à résoudre à la main (marqueurs <<<<<<<, =======, >>>>>>>), puis add + commit.

• GitHub héberge le dépôt distant (origin) : push pour envoyer, clone pour copier, fetch pour regarder sans fusionner, pull = fetch + merge.

• La pull request organise la relecture avant la fusion : description, diff, commentaires, corrections, approbation — la branche main reste protégée.

• Bonnes pratiques : petits commits fréquents, messages clairs (verbe + objet), .gitignore dès le départ, aucun secret dans l'historique, pull avant push.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Pourquoi gérer les versions : historique et collaboration
Un commit est un instantané du projet, identifié par un code unique (le hash), avec un auteur, une date et un message. ; Git est distribué : chaque membre de l'équipe possède l'historique entier du projet, pas seulement la dernière version. ; Git n'est pas GitHub : Git est l'outil local d'historique, GitHub est le service en ligne qui héberge les dépôts et organise la relecture.

Fiche 2 — Commandes de base : init, add, commit et log
git add ne sauvegarde rien : il prépare le prochain commit. C'est git commit qui écrit dans l'historique. ; git status est ton meilleur ami : lance-le avant et après chaque opération pour savoir où en sont tes fichiers. ; Un fichier jamais ajouté est « non suivi » (untracked) : Git le voit, mais il n'entrera dans aucun commit tant qu'il n'est pas dans l'index.

Fiche 3 — Branches : créer, fusionner et résoudre les conflits
Fast-forward ou commit de fusion : dans les deux cas, après git merge, ta branche courante contient tout le travail de la branche fusionnée. ; <<<<<<< HEAD désigne TA branche courante ; >>>>>>> nom désigne la branche que tu es en train de fusionner. ; Un conflit se termine toujours par : éditer le fichier, git add, git commit. Et git merge --abort permet de tout annuler avant la résolution.

Fiche 4 — Travailler avec GitHub : remote, push et pull
git push -u origin main la première fois ; ensuite git push et git pull suffisent, grâce au suivi de branche. ; fetch ne modifie jamais tes fichiers : il met à jour origin/main, la copie de contrôle du distant, que tu peux inspecter avant de fusionner. ; pull = fetch + merge : si ton travail local touche les mêmes lignes que les commits reçus, un conflit peut apparaître, exactement comme pour un merge local.

Fiche 5 — Pull requests et relecture de code entre pairs
Une PR se décrit comme un courrier à l'équipe : quoi, pourquoi, comment tester. Sans description, la relecture est aveugle. ; Les commits correctifs poussés sur la branche mettent à jour la PR automatiquement : inutile d'en ouvrir une deuxième. ; Relire, ce n'est pas chercher la faute : c'est vérifier la logique, la lisibilité et les cas limites — et apprendre le code des autres.

Fiche 6 — Bonnes pratiques : messages de commit et fichiers ignorés
Message de commit : un verbe d'action + l'objet du changement. Le « pourquoi » va dans le corps du message, pas dans le titre. ; Le .gitignore n'efface rien : il empêche Git de suivre les fichiers générés, personnels ou secrets — il doit être committé dès le début du projet. ; merge conserve l'histoire (commit de fusion), rebase la réécrit (historique linéaire) : ne rebase jamais une branche déjà partagée.

Fiche 7 — Projet collaboratif mené en équipe avec Git
Une branche par tâche, une pull request par branche : jamais de développement direct sur main quand on travaille en équipe. ; Fusionner main dans sa branche régulièrement évite le « conflit géant » de la veille de la remise. ; Le graphe git log --graph --oneline --all est la preuve visuelle du travail collectif : montrez-le au jury de soutenance.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Git & GitHub Workshop 2025 - Part 1: Fondamentaux de Git | Zero to Hero en Français

Chaîne : Elias W. BA · Langue : Français · Niveau : Débutant

Un atelier complet qui part de zéro : ce qu'est Git et pourquoi l'utiliser, initialiser un dépôt, faire ses premiers commits et inspecter l'historique avec git log. Il se termine par la connexion du projet à GitHub et les premiers push et pull : le compagnon idéal des deux premières leçons.

https://www.youtube.com/watch?v=cGyc_wv-ekA

Leçon 2 — LES BASES DE GIT (tuto débutant)

Chaîne : Graven - Développement · Langue : Français · Niveau : Débutant

Les bases de Git en une vidéo claire : création du dépôt, clone, ajout de fichiers et commits, puis le travail à plusieurs et le fonctionnement des branches. Parfait pour revoir la leçon 2 et entrevoir la leçon 3 avant de la commencer.

https://www.youtube.com/watch?v=gp_k0UVOYMw

Leçon 4 — Maîtriser Git Bash et les Commandes Git pour Débutants 1H (Tutoriel complet Très Facile)

Chaîne : Âme du Dev Consulting · Langue : Français · Niveau : Débutant

Une heure de pratique dans le terminal : git init, add, commit, log, status, clone, branch, merge, puis la connexion à GitHub avec push, fetch et pull, jusqu'à la résolution d'un conflit de fusion. À suivre en tapant les commandes en même temps que la vidéo.

https://www.youtube.com/watch?v=YPeadnyI450

Leçon 7 — GIT - Versionnez vos projets, en équipe et à distance avec Git et Github

Chaîne : Aziz DAAIF · Langue : Français · Niveau : Débutant à intermédiaire

Versionner en local (init, status, add, commit), puis travailler en équipe à distance grâce à GitHub : branches, merge, remote, pull et push, avec la résolution des conflits lors des fusions. Une bonne synthèse vidéo du projet collaboratif de la leçon 7.

https://www.youtube.com/watch?v=ajQpILOuqcw

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Chacon (Scott) & Straub (Ben) — « Pro Git », 2e édition, livre en accès libre sous licence Creative Commons, version française sur git-scm.com/book/fr : la référence complète sur Git.

• Documentation officielle de Git — git-scm.com/doc : pages de manuel de chaque commande (aussi accessibles en local : git help commit, git help merge…).

• Documentation GitHub — docs.github.com/fr : guides sur les dépôts distants, les pull requests et la relecture de code.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !